

category: dotnet-general

tags: [wsl, linux, dotnet-publish, dpkg-deb, projectreference, msys, git-bash, path-conversion, quoting]

severity: high

created: 2026-07-23

updated: 2026-07-23

project-origin: iPGSv4 (IPGS.Kiosk.Avalonia)

Build .deb qua WSL: ProjectReference tuyệt đối Windows + MSYS path-conv + quoting lồng qua wsl.exe

Tình huống gặp phải

Chạy `scripts/linux-deb/build-deb.sh` (sinh bởi skill `/gen-build-deb`) cho `IPGS.Kiosk.Avalonia` — publish self-contained `linux-x64` rồi đóng gói `.deb` bằng `dpkg-deb` qua WSL Ubuntu, gọi từ Git Bash trên máy dev Windows.

Triệu chứng / Lỗi

Ba lỗi liên tiếp xuất hiện khi chạy script:

1. `dotnet publish` qua dotnet trong WSL báo lỗi build:

```
Skipping project ".../IPGS.Kiosk.Avalonia/E:/KZTEK/.../Kztek.Tool.MultyPlatform.csproj" because it was not found.
```

```
error CS0246: The type or namespace name 'iParkingv8' could not be found
```

```
error CS0234: The type or namespace name 'Tool' does not exist in the namespace 'Kztek'
```

2. Sau khi tách bước packaging ra gọi `wsl.exe -- bash -lc "<script nhiều dòng có heredoc>"`:

```
du: cannot access "/opt/kztek/ipgskioskavalonia": No such file or directory
```

```
mkdir: Permission denied
```

3. Sau khi tách packaging ra file riêng `_pack-deb-inner.sh` và gọi `wsl.exe -- bash "$path_wsl" ...`:

```
bash: C:/Program Files/SetUp/Git/mnt/e/KZTEK/.../scripts/linux-deb/_pack-deb-inner.sh: No such file or directory
```

Nguyên nhân gốc rễ (Root Cause)

Ba nguyên nhân độc lập, xếp lớp lên nhau:

1. **ProjectReference tuyệt đối kiểu Windows không tương thích dotnet Linux.** `.csproj` có `<ProjectReference Include="E:\KZTEK\...\ParkingV8.UI.csproj" />` — bắt buộc vì 2 repo song song trên máy dev. `dotnet` chạy trong WSL (Linux) không hiểu `E:\...` là path tuyệt đối (không có ổ đĩa, backslash không phải separator) → MSBuild nổi nhằm thành path con của project hiện tại và "Skip" — kéo theo hàng loạt lỗi thiếu type/namespace ở downstream, dễ nhầm là lỗi thiếu package.
2. **Quoting lồng 2 lớp qua `wsl.exe -- bash -lc "<script nhiều dòng>"`.** Đưa cả script nhiều dòng (có heredoc `<<CTRL`, biến `\$VAR` escape, dấu `'/'` xen nhau) vào MỘT chuỗi double-quote của lệnh outer bash → biến outer bash tưởng cần expand lại expand nhầm, escape lệch 1 lớp → biến đích (`PKG_ROOT`) rỗng ở phía WSL, path build thành rỗng + ký tự lạ.
3. **MSYS/Git Bash tự dịch path bắt đầu bằng `/` sang path Windows khi gọi native exe.** Git Bash phát hiện `wsl.exe` là native Windows executable (không phải MSYS binary) nên tự "helpfully" dịch argument `/mnt/e/KZTEK/...` thành `C:/Program Files/SetUp/Git/mnt/e/KZTEK/...` (chèn prefix root cài Git) trước khi thực thi — hành vi ẩn, không có log cảnh báo.

Giải pháp

1. **Publish bằng `dotnet.exe` gốc trên Windows** (Git Bash/PowerShell) — nơi path `E:\...` hợp lệ — thay vì `dotnet` trong WSL. Chỉ dùng WSL cho bước cần Linux thật (`dpkg-deb`, `chmod`, `symlink`).

```
``bash
```

```
dotnet publish "$APP_PROJECT" -c Release -r linux-x64 --self-contained true -o "$PUBLISH_DIR"
```

```
`
```

2. **Tách phần script nhiều dòng ra file `.sh` riêng**, gọi qua `wsl.exe -- bash "<path>" <args...>` — KHÔNG nhồi script vào 1 chuỗi `bash -lc "..."`. Tránh hoàn toàn vấn đề quoting lồng lớp.
3. **Set `MSYS_NO_PATHCONV=1`** ngay trước lệnh gọi `wsl.exe` (hoặc bất kỳ native exe Windows nào nhận argument dạng POSIX path từ Git Bash):

```
``bash
```

```
MSYS_NO_PATHCONV=1 wsl.exe -d Ubuntu -- bash "$inner_script_wsl" "$arg1" "$arg2" ...
```

```
`
```

4. Copy publish output (đã publish trên Windows, nằm ở `/mnt/<drive>/...` khi nhìn từ WSL) vào filesystem Linux thật (`/tmp/...` trong WSL) TRƯỚC khi `dpkg-deb --build` — không build trực tiếp trên `/mnt/*` (gotcha DrvFs permission/symlink đã biết từ trước).

Áp dụng lại (How to reuse)

- Thấy `dotnet publish/build` trong WSL báo thiếu type/namespace ở project mà build Windows bình thường không lỗi → kiểm tra ngay `.csproj` có `ProjectReference` dùng path tuyệt đối Windows (`X:\...`) không, trước khi nghi ngờ package/SDK.
- Bất kỳ khi nào cần gọi `wsl.exe` (hoặc exe Windows native khác) từ Git Bash với một script nhiều dòng → viết ra file `.sh` riêng rồi gọi `bash <file> <args>`, không truyền script inline qua `-c/-lc` với chuỗi dài có heredoc/escape.

- Bất kỳ khi nào truyền path dạng `/mnt/...` hoặc `/...` cho native exe Windows từ Git Bash → luôn set `MSYS_NO_PATHCONV=1` trước lệnh đó, kiểm tra bằng cách echo lại path nhận được ở phía đích nếu nghi ngờ.
- Khi 1 project cần build cross-environment (Windows dotnet publish → Linux dpkg-deb) mà có ProjectReference tuyệt đối: publish luôn trên OS mà path đó hợp lệ, chỉ chuyển sang OS khác cho bước không thể làm khác (đóng gói định dạng OS-specific).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `MSYS_NO_PATHCONV=1` chỉ tắt dịch path cho lệnh ngay sau nó (khi đặt làm prefix của 1 lệnh) — không set global trừ khi chắc chắn không ảnh hưởng lệnh Git Bash khác cần dịch path đúng (VD gọi `.exe` khác cần path Windows thật).
- ⚠ Lỗi "No such file or directory" với path bị chèn thêm `C:/Program Files/.../Git` phía trước rất dễ nhầm là lỗi mount WSL hoặc quyền — luôn echo lại path ngay trước khi gọi native exe để phát hiện path-conv sớm.
- ⚠ Sau khi publish trên Windows, thư mục publish nằm trên `/mnt/<drive>` khi nhìn từ WSL — vẫn phải `cp -a` sang `/tmp` (Linux fs thật) trước `dpkg-deb --build`, không bỏ qua bước này chỉ vì "đã có file rồi".
- ⚠ Đừng cố sửa `.csproj` đổi ProjectReference tuyệt đối thành tương đối để "cho dễ build WSL" nếu 2 repo không thực sự nằm cùng cấu trúc tương đối trên mọi máy dev — rủi ro làm hỏng build Windows hiện tại của người khác; giải pháp build-script (bài này) an toàn hơn sửa cấu trúc project.

Tham chiếu

- Project liên quan: `iPGSv4` — `scripts/linux-deb/build-deb.sh`, `scripts/linux-deb/_pack-deb-inner.sh`, `IPGS.Kiosk.Avalonia/IPGS.Kiosk.Avalonia.csproj`
- Lesson liên quan: `[wsl-linux-deb-build-global-json-sdk-version-mismatch.md]` (`wsl-linux-deb-build-global-json-sdk-version-mismatch.md`) — cùng chủ đề build `.deb` qua WSL, gotcha khác (SDK version)
- Skill liên quan: `.claude/commands/gen-build-deb.md`